

Data Administration in Information Systems

Project Report – 2025/2026

Rodrigo Perestrelo - ist1106074

João Santos - ist1107365

April 2, 2026

Contents

1	Task 1: Table Creation and Data Insertion	1
1.1	Data Insertion via BULK INSERT	1
1.2	Results	4
2	Task 2: Storage and Partitioning	4
2.1	Choice of Number of Partitions	4
2.2	Distribution Analysis by County	4
2.3	Creation of the Partition Function and Scheme	5
2.4	Partition Verification	5
3	Task 3: Indexes and Query Cost	6
3.1	No Index - Baseline (SubTree Cost 10.0607)	6
3.2	Simple Index on <code>town</code> (SubTree Cost 10.0595)	7
3.3	Composite Index on <code>town</code> and <code>OldNew</code> (SubTree Cost 4.67951)	7
3.4	Index with <code>locality</code> in the key and <code>INCLUDE</code> of <code>price</code> (SubTree Cost 0.0297144)	8
3.5	Index with <code>locality</code> in <code>INCLUDE</code> (SubTree Cost 0.0656608)	9
3.6	Comparative Summary	9
4	Task 4: Study of Join and Aggregation Algorithms	9
4.1	Combinations with Hash Group (Hash Aggregation)	10
4.1.1	Hash Aggregate + Hash Join (Optimal Configuration)	10
4.1.2	Hash Aggregate + Nested Loop Join	11
4.1.3	Hash Aggregate + Merge Join	12
4.2	Combinations with Order Group (Stream Aggregation)	13
4.2.1	Stream Aggregate + Hash Join	13
4.2.2	Stream Aggregate + Merge Join (Worst Case)	14
4.2.3	Stream Aggregate + Loop Join	15
5	Task 5: Query Tuning	16
6	Task 6: Transactions and Isolation Levels	17
6.1	Scenario 1 - READ UNCOMMITTED: Dirty Read and Non-Repeatable Read	17
6.2	Scenario 2 - READ COMMITTED: Dirty Read Prevention	18
6.3	Scenario 3 - SERIALIZABLE: Maximum Isolation	19

1 Task 1: Table Creation and Data Insertion

In this task, the storage tables were created and data from CSV files was loaded.

The tables `ShortPricePaidData2024` and `ShortPricePaidData2025` were created with the 12 columns specified in the assignment, including integrity constraints (`CHECK`) to validate `PropertyType` and `OldNew`:

Listing 1: Main tables creation

```
CREATE TABLE ShortPricePaidData2024 (  
    TransactionID UNIQUEIDENTIFIER DEFAULT NEWID(),  
    Price MONEY NOT NULL,  
    DateOfTransfer DATETIME NOT NULL,  
    Postcode VARCHAR(10),  
    PropertyType CHAR(1) NOT NULL,  
    OldNew CHAR(1) NOT NULL,  
    PAON VARCHAR(60) NOT NULL,  
    Street VARCHAR(60),  
    Locality VARCHAR(40),  
    Town VARCHAR(40),  
    District VARCHAR(40),  
    County VARCHAR(40),  
  
    CONSTRAINT CK_PropertyType CHECK (PropertyType IN ('D', 'S', 'T', 'F', 'O')),  
    CONSTRAINT CK_OldNew CHECK (OldNew IN ('Y', 'N'))  
);  
  
CREATE TABLE ShortPricePaidData2025 (  
    TransactionID UNIQUEIDENTIFIER DEFAULT NEWID(),  
    Price MONEY NOT NULL,  
    DateOfTransfer DATETIME NOT NULL,  
    Postcode VARCHAR(10),  
    PropertyType CHAR(1) NOT NULL,  
    OldNew CHAR(1) NOT NULL,  
    PAON VARCHAR(60) NOT NULL,  
    Street VARCHAR(60),  
    Locality VARCHAR(40),  
    Town VARCHAR(40),  
    District VARCHAR(40),  
    County VARCHAR(40)  
);
```

1.1 Data Insertion via BULK INSERT

Since the CSV files contain 16 columns in total (including fields such as `Duration`, `SAON`, `PPD_Category` and `Record_Status` that are not required), the following strategy was adopted: Create a *staging* table with all CSV columns, load the file using `BULK INSERT`, copy only the 12 relevant columns into the final table, and drop the *staging* table.

Listing 2: Complete insertion pipeline (example for 2024)

```
-- Phase 1: Staging table with full CSV structure
```

```
CREATE TABLE PricePaidData2024 (  
    TransactionID UNIQUEIDENTIFIER,  
    Price MONEY,  
    DateOfTransfer DATETIME,  
    Postcode VARCHAR(10),  
    PropertyType CHAR(1),  
    OldNew CHAR(1),  
    Duration CHAR(1),  
    PAON VARCHAR(60),  
    SAON VARCHAR(40),  
    Street VARCHAR(60),  
    Locality VARCHAR(40),  
    Town VARCHAR(40),  
    District VARCHAR(40),  
    County VARCHAR(40),  
    PPD_Category CHAR(1),  
    Record_Status CHAR(1)  
);
```

```
CREATE TABLE PricePaidData2025 (  
    TransactionID UNIQUEIDENTIFIER,  
    Price MONEY,  
    DateOfTransfer DATETIME,  
    Postcode VARCHAR(10),  
    PropertyType CHAR(1),  
    OldNew CHAR(1),  
    Duration CHAR(1),  
    PAON VARCHAR(60),  
    SAON VARCHAR(40),  
    Street VARCHAR(60),  
    Locality VARCHAR(40),  
    Town VARCHAR(40),  
    District VARCHAR(40),  
    County VARCHAR(40),  
    PPD_Category CHAR(1),  
    Record_Status CHAR(1)  
);
```

```

-- Phase 2: Bulk load from CSV file
BULK INSERT PricePaidData2024
FROM 'C:\Users\Public\Downloads\pp-2024.csv'
WITH (
    FORMAT = 'CSV', FIRSTROW = 1,
    FIELDTERMINATOR = ',', ROWTERMINATOR = '0x0a',
    ERRORFILE = 'C:\Users\Public\Downloads\import_errors.log',
    TABLOCK
);

BULK INSERT PricePaidData2025
FROM 'C:\Users\Public\Downloads\pp-2025.csv'
WITH (
    FORMAT = 'CSV',
    FIRSTROW = 1,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    ERRORFILE = 'C:\Users\Public\Downloads\import_errors.log',
    TABLOCK
);

-- Phase 3: Copy only the 12 relevant columns into the final table
INSERT INTO ShortPricePaidData2024 (
    TransactionID, Price, DateOfTransfer, Postcode,
    PropertyType, OldNew, PAON, Street,
    Locality, Town, District, County
)
SELECT
    TransactionID, Price, DateOfTransfer, Postcode,
    PropertyType, OldNew, PAON, Street,
    Locality, Town, District, County
FROM PricePaidData2024;

INSERT INTO ShortPricePaidData2025 (
    TransactionID, Price, DateOfTransfer, Postcode, PropertyType,
    OldNew, PAON, Street, Locality, Town, District, County
)
SELECT
    TransactionID, Price, DateOfTransfer, Postcode, PropertyType,
    OldNew, PAON, Street, Locality, Town, District, County
FROM PricePaidData2025;

-- Phase 4: Drop the staging table
DROP TABLE PricePaidData2024;

DROP TABLE PricePaidData2025;

```

Listing 3: Verification of the number of records in the final tables

```
SELECT COUNT(*) AS TotalRecords FROM ShortPricePaidData2024;
-- Result: 921,562 records

SELECT COUNT(*) AS TotalRecords FROM ShortPricePaidData2025;
-- Result: 757,816 records
```

1.2 Results

Table	Number of Records
ShortPricePaidData2024	921 562
ShortPricePaidData2025	757 816

Table 1: Number of tuples loaded into each table after insertion

2 Task 2: Storage and Partitioning

In this task, a partitioning scheme was implemented on the ShortPricePaidData2025 table, dividing records into distinct partitions according to the County column.

2.1 Choice of Number of Partitions

We considered **5 partitions** to be an appropriate number for this dataset, as it represents a reasonable balance between distribution granularity and the complexity of managing the partitioning scheme. With a total of 757 816 records, each partition would ideally contain $757\,816 \div 5 \approx 151\,563$ records.

2.2 Distribution Analysis by County

To define balanced partition boundaries, we started by obtaining the record count per county and the cumulative total sorted alphabetically:

Listing 4: Record count and cumulative total by County

```
SELECT
    County,
    record_count,
    SUM(record_count) OVER (ORDER BY County) AS CumulativeTotal
FROM (
    SELECT County, COUNT(*) AS record_count
    FROM ShortPricePaidData2025
    GROUP BY County
) AS Sub
ORDER BY County;
```

Based on the results of this query, the `County` values whose cumulative total most closely approached the multiples of $\approx 151\,563$ (i.e., $\approx 151\,563$, $\approx 303\,126$, $\approx 454\,689$ and $\approx 606\,252$) were identified. These values were used as the boundaries of the *Partition Function*, resulting in the boundaries `EAST SUSSEX`, `GREATER MANCHESTER`, `NEWPORT` and `SURREY`.

2.3 Creation of the Partition Function and Scheme

`RANGE RIGHT` was used, each boundary value is included in the right-hand partition (the membership condition is $\geq$ boundary for the next partition).

Listing 5: Partition Function creation with 4 boundaries

```
CREATE PARTITION FUNCTION PF_County (VARCHAR(100))
AS RANGE RIGHT FOR VALUES (
    'EAST SUSSEX',
    'GREATER MANCHESTER',
    'NEWPORT',
    'SURREY'
);
```

Note: In a real scenario, each partition should be mapped to a physically separate *filegroup* (on distinct disks) to maximise parallelism. In this context, all partitions are mapped to the *filegroup* `[PRIMARY]`.

2.4 Partition Verification

To confirm that the data was correctly distributed across the partitions, the following system query was executed:

Listing 6: Verification of the distribution by partition

```
SELECT
    p.partition_number AS [Partition],
    fg.name AS [Filegroup],
    p.rows AS [Num Records]
FROM sys.partitions AS p
JOIN sys.destination_data_spaces AS dds
    ON p.partition_number = dds.destination_id
JOIN sys.filegroups AS fg
    ON dds.data_space_id = fg.data_space_id
WHERE p.object_id = OBJECT_ID('ShortPricePaidData2025')
    AND p.index_id <= 1
ORDER BY p.partition_number;
```

The instances are distributed across the partitions as follows:

	Nº da Partição	Filegroup	Nº de Registos
1	1	PRIMARY	147736
2	2	PRIMARY	122685
3	3	PRIMARY	176692
4	4	PRIMARY	148227
5	5	PRIMARY	162476

Figure 1: Verification query result: record distribution across the 5 partitions

3 Task 3: Indexes and Query Cost

In this task, the impact of different indexing strategies on the *Estimated Subtree Cost* of the following query was analysed. The query calculates the average price of new properties by locality in London:

Listing 7: Reference query for index analysis

```
SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
WHERE town = 'London'
AND OldNew = 'Y'
GROUP BY locality
ORDER BY average_new_property_price DESC;
```

3.1 No Index - Baseline (SubTree Cost 10.0607)

Without any auxiliary index, SQL Server performs a full *Table Scan*, reading all 757 816 records in the table before applying the *WHERE* filters. The absence of a selective access path forces the engine to scan the entire table.

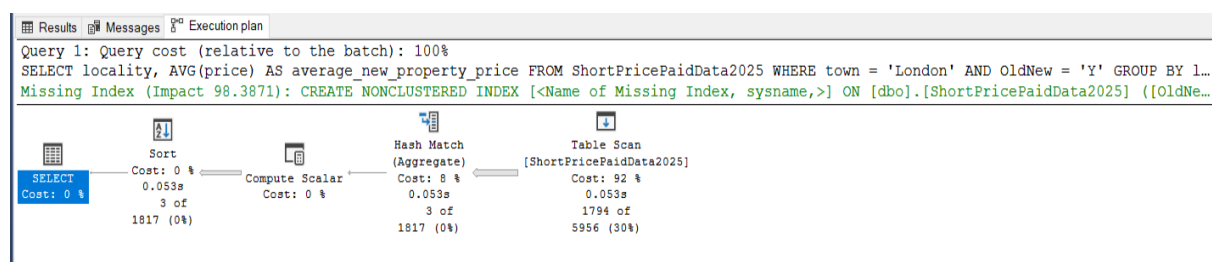


Figure 2: Execution plan without index (Subtree Cost: 10.0607)

3.2 Simple Index on town (SubTree Cost 10.0595)

Listing 8: Simple non-clustered index on town

```
CREATE INDEX idx_town ON ShortPricePaidData2025(town);

-- Query to drop the respective index
DROP INDEX idx_town ON ShortPricePaidData2025;
```

With an index only on `town` on a heap table, the optimiser determines that a *Non-Clustered Index Seek* followed by individual *RID Lookups* into the heap for each matching row would be more expensive than simply scanning all data pages directly. As a result, SQL Server ignores the index and once again performs a full *Table Scan*, leaving the cost virtually unchanged from the *baseline*.

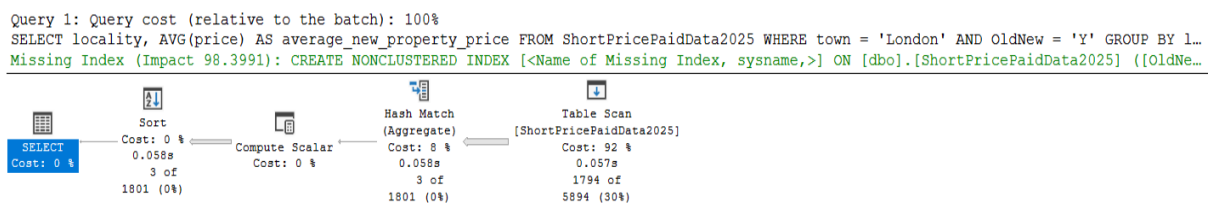


Figure 3: Execution plan with index on `town` (Subtree Cost: 10.0595)

3.3 Composite Index on town and OldNew (SubTree Cost 4.67951)

Listing 9: Composite non-clustered index on town and OldNew

```
CREATE INDEX idx_town_oldnew ON ShortPricePaidData2025(town, OldNew);

-- Query to drop the respective index
DROP INDEX idx_town_oldnew ON ShortPricePaidData2025;
```

By including `OldNew` in the index keys, SQL Server can filter simultaneously by `town = 'London'` and `OldNew = 'Y'` during the *Seek*, substantially reducing the number of rows that require an *RID Lookup (Heap)* into the data pages. The cost drops by $\approx 53\%$ compared to the *baseline*.

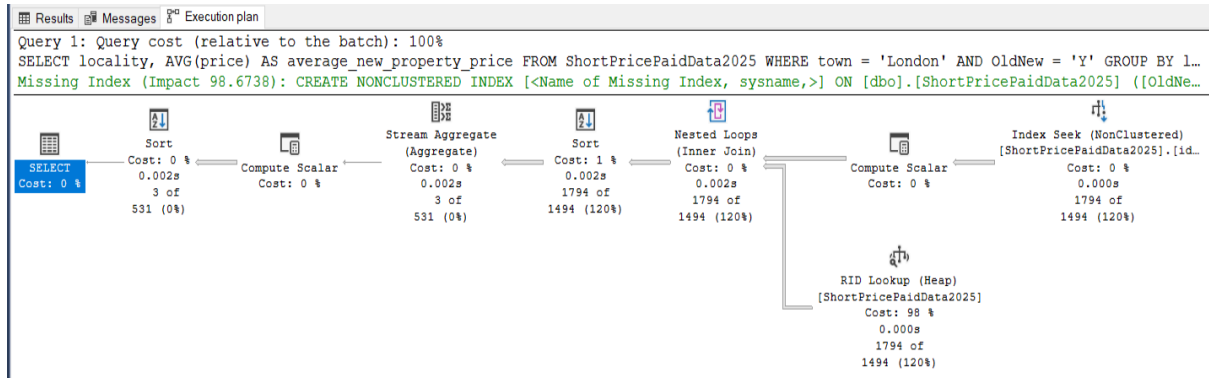


Figure 4: Execution plan with index on (town, OldNew) (Subtree Cost: 4.67951)

3.4 Index with locality in the key and INCLUDE of price (SubTree Cost 0.0297144)

Listing 10: Covering index with locality in the key and price in INCLUDE

```
CREATE INDEX idx_query_cover
ON ShortPricePaidData2025(town, OldNew, locality)
INCLUDE(price);

-- Query to drop the respective index
DROP INDEX idx_query_cover ON ShortPricePaidData2025;
```

This index completely covers the query: the filter columns (town, OldNew), the grouping column (locality), and the calculated column (price) are all present in the index. SQL Server eliminates the *RID Lookup*, resulting in a $\approx 99.7\%$ reduction compared to the *baseline*. Furthermore, since *locality* is stored as an index key, the data returned by the *Seek* is already ordered by *locality*, enabling a *Stream Aggregate* without a pre-sort step. This index was suggested by SQL Server through the *Missing Index* warning in the previous plans.

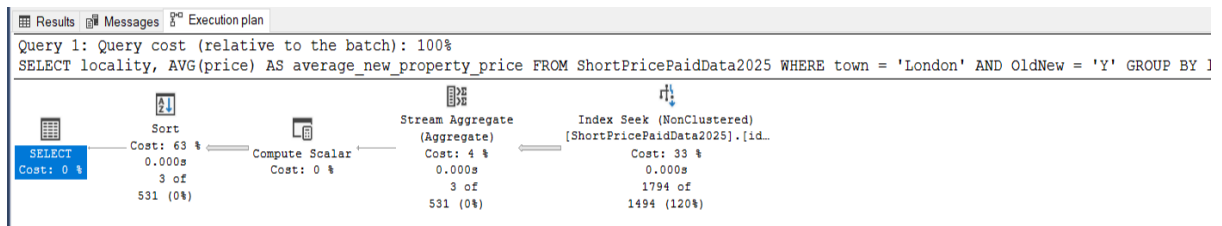


Figure 5: Execution plan with covering index (town, OldNew, locality) INCLUDE(price) (Subtree Cost: 0.0297144)

3.5 Index with locality in INCLUDE (SubTree Cost 0.0656608)

Listing 11: Optimal covering index with locality in INCLUDE

```
CREATE INDEX idx_query_cover_2
ON ShortPricePaidData2025(town, OldNew)
INCLUDE(price, locality);

-- Query to drop the respective index
DROP INDEX idx_query_cover_2 ON ShortPricePaidData2025;
```

Moving *locality* to *INCLUDE* makes the leaf pages more compact, but it removes the ordering guarantee on *locality*. As a result, SQL Server must inject an additional *Sort* operator before the *Stream Aggregate* (to satisfy the *GROUP BY*) and a second *Sort* for the *ORDER BY* clause. These two sort passes cause this configuration to cost $\approx 121\%$ **more** than *idx_query_cover*, making *idx_query_cover* (with *locality* as a key column) the optimal choice for this query.

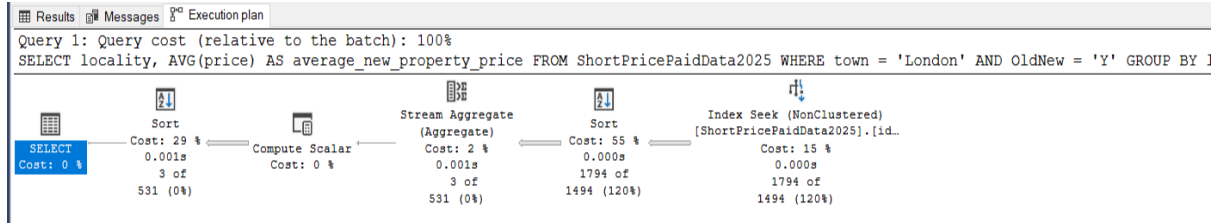


Figure 6: Optimal execution plan with (town, OldNew) INCLUDE(price, locality) (Subtree Cost: 0.0656608)

3.6 Comparative Summary

Configuration	Subtree Cost	Reduction
No index (baseline)	10.0607	-
idx_town (town)	10.0595	$\approx 0\%$
idx_town.oldnew (town, OldNew)	4.67951	$\approx 53\%$
idx_query_cover (town, OldNew, loc.)	0.0297144	$\approx 99.7\%$
INC(price)		
idx_query_cover_2 (town, OldNew)	0.0656608	$\approx 99.3\%$
INC(price, loc.)		

Table 2: Comparison of the Estimated Subtree Cost by index configuration

4 Task 4: Study of Join and Aggregation Algorithms

In this task, a comparative study of the *Estimated Subtree Cost* was conducted for the query that identifies districts with an increase in average prices between 2024 and 2025. Combinations of aggregation algorithms (*Hash* vs *Order/Stream*) and join algorithms (*Hash*, *Merge*, *Loop*) were tested using *Query Hints*.

The analysed query identifies districts where the average price increased between 2024 and 2025:

Listing 12: Base query without hints (automatic optimiser choice)

```
SELECT p24.district, p24.avg_price AS avg_2024, p25.avg_price AS avg_2025
FROM ( SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price
      FROM ShortPricePaidData2024 GROUP BY district ) p24
JOIN ( SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price
      FROM ShortPricePaidData2025 GROUP BY district ) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price;
```

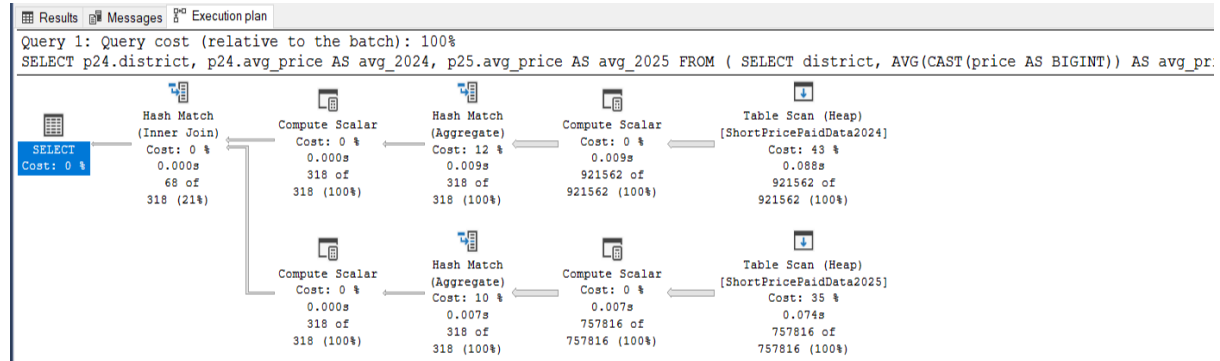


Figure 7: Execution plan without hints: the optimiser automatically chooses Hash Aggregate + Hash Join (Subtree Cost: 26.225)

Without any hint, the optimiser automatically chooses the Hash Aggregate + Hash Join combination, the most efficient for unordered data.

4.1 Combinations with Hash Group (Hash Aggregation)

In this configuration, SQL Server uses in-memory hash tables for grouping, which is efficient for unordered data.

4.1.1 Hash Aggregate + Hash Join (Optimal Configuration)

```
SELECT p24.district, p24.avg_price AS avg_2024, p25.avg_price AS avg_2025
FROM ( SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2024 GROUP BY district ) p24
JOIN ( SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2025 GROUP BY district ) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (HASH GROUP, HASH JOIN);
```

Analysis (SubTree Cost 26.225) - Optimal Configuration: This combination achieved the lowest estimated cost. Since the source tables are not pre-sorted by *district*, the optimiser creates in-memory hash tables for both the grouping and the join phases.

Why it performs better: It completely avoids the *Sort* operator. By using in-memory hash structures to map values, it maintains a linear scan cost instead of the $O(n \log n)$ cost associated with sorting large datasets.

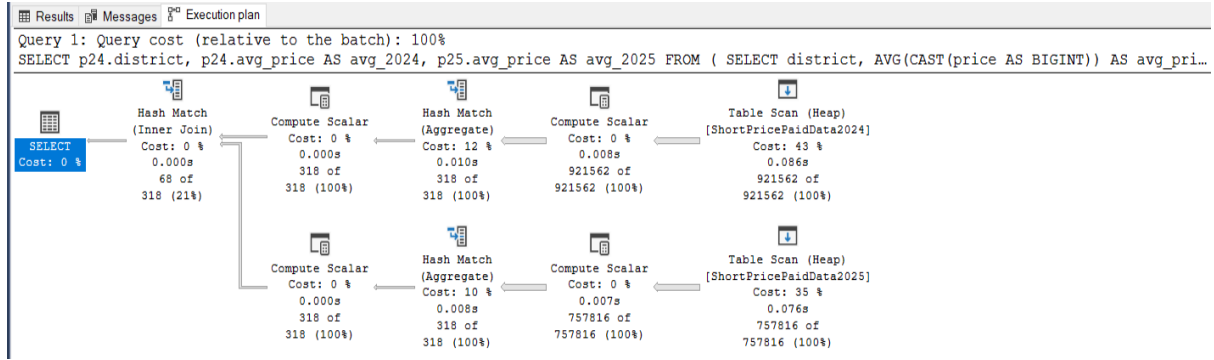


Figure 8: Execution plan: Hash Aggregate + Hash Join (Subtree Cost: 26.225)

4.1.2 Hash Aggregate + Nested Loop Join

```
SELECT p24.district, p24.avg_price, p25.avg_price
FROM (SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2024 GROUP BY district) p24
JOIN (SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2025 GROUP BY district) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (HASH GROUP, LOOP JOIN);
```

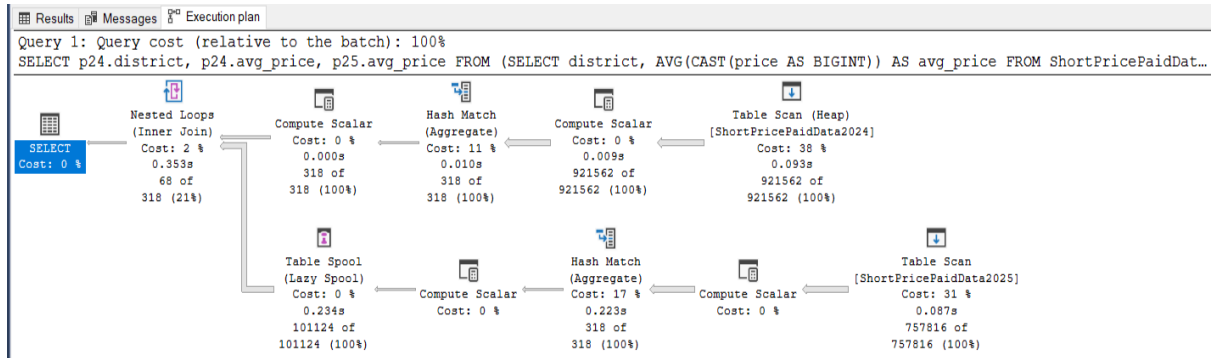


Figure 9: Execution plan: Hash Aggregate + Nested Loop Join (Subtree Cost: 29.3477)

Analysis (SubTree Cost 29.3477) - Moderate Performance: Grouping is performed efficiently via hash, but the join uses *Nested Loop* logic.

Plan detail: The plan includes a *Table Spool (Lazy Spool)* operator, used to cache the 2025 aggregation results and avoid a full re-scan of the sub-query for each row found in 2024. The overhead of managing this spool makes this combination slightly more expensive than a pure *Hash Join*.

4.1.3 Hash Aggregate + Merge Join

```
SELECT p24.district, p24.avg_price, p25.avg_price
FROM (SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2024 GROUP BY district) p24
JOIN (SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2025 GROUP BY district) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (HASH GROUP, MERGE JOIN);
```

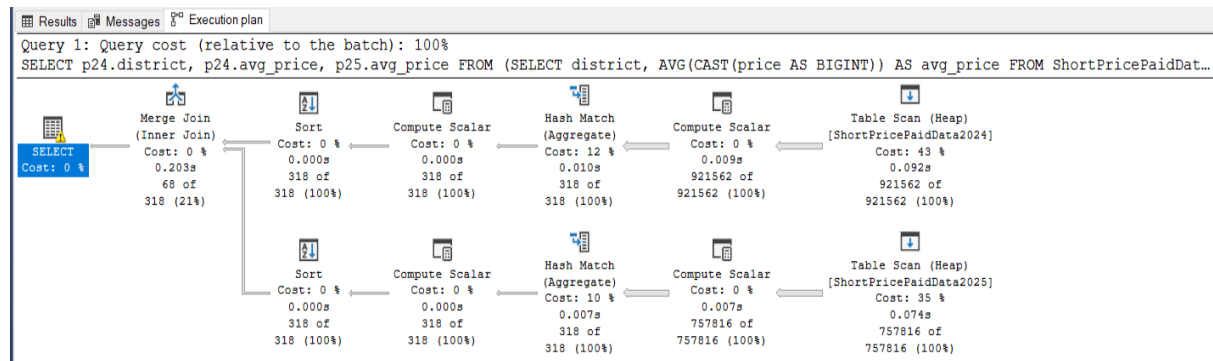


Figure 10: Execution plan: Hash Aggregate + Merge Join (Subtree Cost: 26.2311)

Analysis (SubTree Cost 26.2311) - Poor Performance: This combination creates a “bottleneck” effect. Although the initial grouping is fast via hash, the *Merge Join* strictly requires sorted inputs.

Why it performs worse: The optimiser is forced to inject *Sort* operators after the aggregation is complete, in order to prepare the data for the join. This transition from “hashed” data back to “sorted” data introduces unnecessary computational overhead.

4.2 Combinations with Order Group (Stream Aggregation)

Order Group requires that the data arrives at the aggregation operator already sorted by *district*.

4.2.1 Stream Aggregate + Hash Join

```
SELECT p24.district, p24.avg_price, p25.avg_price
FROM (SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2024 GROUP BY district) p24
JOIN (SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2025 GROUP BY district) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (ORDER GROUP, HASH JOIN);
```

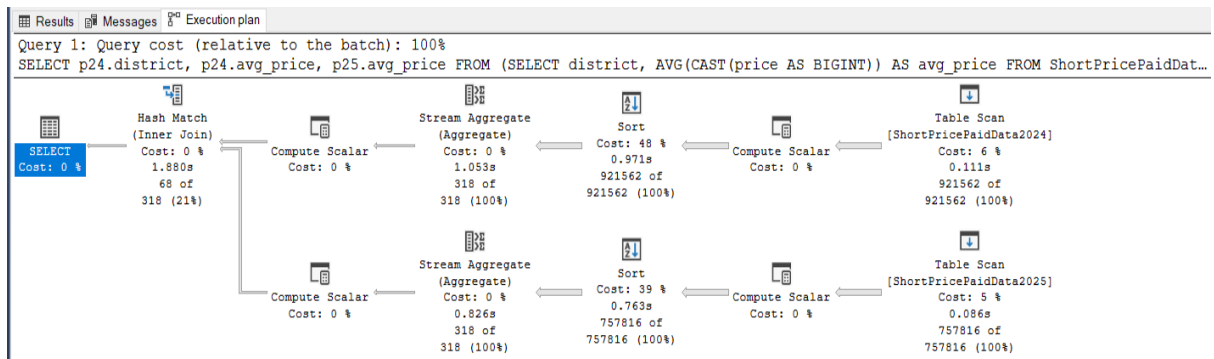


Figure 11: Execution plan: Stream Aggregate + Hash Join (Subtree Cost: 173.47)

Analysis (SubTree Cost 173.47) - Poor Performance: By forcing `ORDER GROUP`, SQL Server must ensure that the data arrives at the aggregation operator already sorted by *district*.

Plan detail: The plan shows heavy *Sort* operators immediately after the *Table Scans (Heap)*. As these sorts operate on the raw data (hundreds of thousands of rows), the *Sort* cost far outweighs any benefit provided by the subsequent *Hash Join*.

4.2.2 Stream Aggregate + Merge Join (Worst Case)

```
SELECT p24.district, p24.avg_price AS avg_2024, p25.avg_price AS avg_2025
FROM ( SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2024 GROUP BY district ) p24
JOIN ( SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2025 GROUP BY district ) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (ORDER GROUP, MERGE JOIN);
```

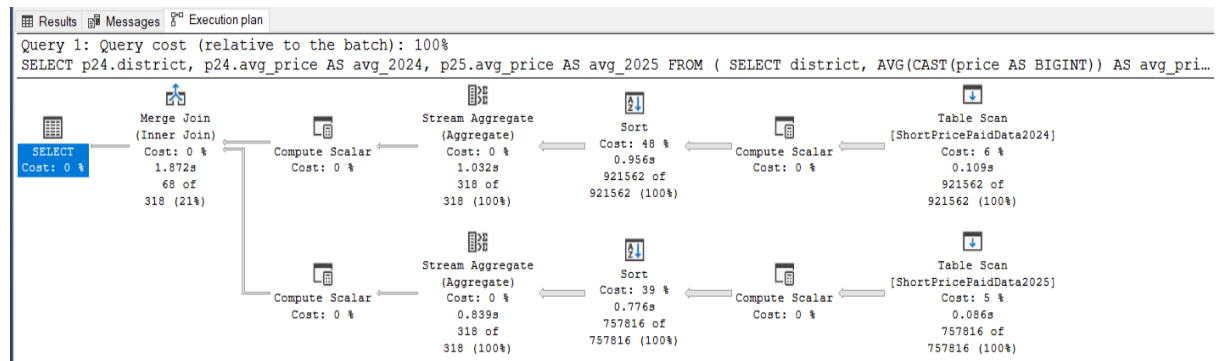


Figure 12: Execution plan: Stream Aggregate + Merge Join (Subtree Cost: ≈ 173.448)

Analysis (SubTree Cost 173.448) - Worst Case: This combination resulted in the highest estimated cost. Both *Stream Aggregate* and *Merge Join* require sorted data.

Conclusion: This is the “Worst Case” for data without an index. The engine must perform massive sorts on both tables. Although the join and aggregation logic is very efficient once the data is sorted, the *Sort* operator becomes the dominant resource consumer in the entire query.

4.2.3 Stream Aggregate + Loop Join

```
SELECT p24.district, p24.avg_price, p25.avg_price
FROM (SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2024 GROUP BY district) p24
JOIN (SELECT district, AVG(CAST(price AS BIGINT)) AS avg_price FROM
      ShortPricePaidData2025 GROUP BY district) p25
ON p24.district = p25.district
WHERE p25.avg_price > p24.avg_price
OPTION (ORDER GROUP, LOOP JOIN);
```

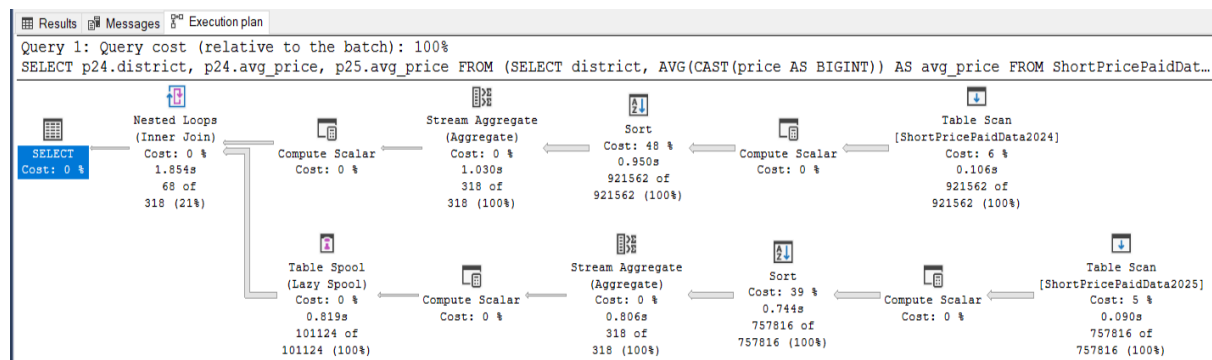


Figure 13: Execution plan: Stream Aggregate + Nested Loop Join (Subtree Cost: 173.994)

Analysis (SubTree Cost 173.994) - Very Poor Performance: Similar to the previous case, the cost is inflated by the initial sort required by the *Stream Aggregate*. Adding a *Nested Loop* over sorted data provides no performance benefit in this context.

Additional note: The engine cannot leverage the existing sort order to execute a faster join, unlike what would happen with a *Merge Join*, where the pre-sorted data would be reused by the join logic.

5 Task 5: Query Tuning

The original query was rewritten to improve performance in identifying properties sold in both years, using the EXISTS clause.

Listing 13: Default Query

```
WITH p24 AS (
SELECT DISTINCT county, PAON, street, town
FROM ShortPricePaidData2024
),
p25 AS (
SELECT DISTINCT county, PAON, street, town
FROM ShortPricePaidData2025
)
SELECT p24.county, COUNT(*) AS properties_sold_both_years
FROM p24 JOIN p25
ON p24.county = p25.county
AND p24.PAON = p25.PAON
AND p24.street = p25.street
AND p24.town = p25.town
GROUP BY p24.county;
```

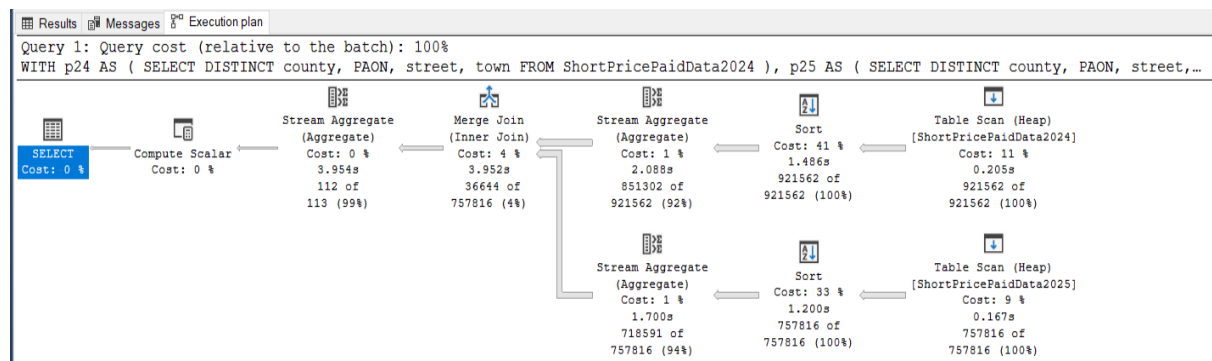


Figure 14: Execution plan of the default query (**Estimated Subtree Cost: 102.124**)

The default query materialises and fully joins deduplicated sets, processing all matching rows (including duplicates), whereas the optimized version uses EXISTS (a semi-join) to match a single instance of each 2024 row.

Listing 14: Optimised Query (Estimated Subtree Cost: 48.8274)

```
SELECT p24.county, COUNT(DISTINCT p24.PAON + p24.street + p24.town) AS
properties_sold_both_years
FROM ShortPricePaidData2024 p24
WHERE EXISTS (
    SELECT 1 FROM ShortPricePaidData2025 p25
    WHERE p24.county = p25.county AND p24.PAON = p25.PAON
    AND p24.street = p25.street AND p24.town = p25.town
)
GROUP BY p24.county
ORDER BY p24.county;
```

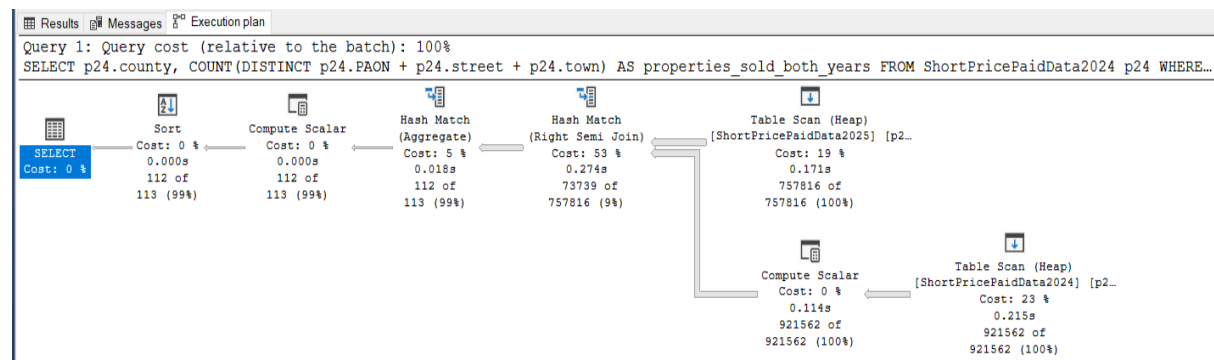


Figure 15: Execution plan of the optimised query (Estimated Subtree Cost: 48.8274)

Justification: EXISTS enables a *Semi-Join*, where processing stops as soon as the first match is found in 2025 for each 2024 record, reducing I/O load compared to a traditional *Join*.

6 Task 6: Transactions and Isolation Levels

Demonstration of how isolation levels affect the correctness of analytical queries running concurrently with insertions. Three scenarios are presented, each executed across two concurrent terminals.

6.1 Scenario 1 - READ UNCOMMITTED: Dirty Read and Non-Repeatable Read

Step 1 [Terminal 1]: Start an insertion but do not commit it.

```
BEGIN TRANSACTION;
INSERT INTO ShortPricePaidData2025 (
    TransactionID, Price, DateOfTransfer, Postcode,
    PropertyType, OldNew, PAON, Street, Locality, Town, District, County
)
VALUES (NEWID(), 5000000, GETDATE(), 'SW1A 1AA', 'D', 'Y',
    '1', 'Downing St', 'Westminster', 'London', 'London', 'Greater London');
```

Step 2 [Terminal 2]: Read the data under READ UNCOMMITTED. The average price for Westminster is drastically inflated by the uncommitted £5 000 000 row.

```

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
BEGIN TRANSACTION;
SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
WHERE town = 'London' AND OldNew = 'Y'
GROUP BY locality;

```

Step 3 [Terminal 1]: Roll back the insertion, the uncommitted row never existed.

```

ROLLBACK TRANSACTION;

```

Step 4 [Terminal 2]: Re-read the data. The Westminster average now returns to its true value, demonstrating a *Dirty Read* followed by a *Non-Repeatable Read* within the same transaction.

```

SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
WHERE town = 'London' AND OldNew = 'Y'
GROUP BY locality;
COMMIT TRANSACTION;

```

READ UNCOMMITTED does not prevent *Dirty Reads* or *Non-Repeatable Reads*. A transaction can read data that has been written but not yet committed by another transaction.

6.2 Scenario 2 - READ COMMITTED: Dirty Read Prevention

Step 1 [Terminal 1]: Start an update but do not commit it.

```

BEGIN TRANSACTION;
UPDATE ShortPricePaidData2025
SET Price = Price * 2
WHERE Locality = 'LEYTON' AND OldNew = 'Y';

```

Step 2 [Terminal 2]: Attempt to read under READ COMMITTED. The query blocks until Terminal 1 releases its locks, or reads only previously committed values, the doubled prices are never visible.

```

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRANSACTION;
SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
WHERE town = 'London' AND OldNew = 'Y'
GROUP BY locality;

```

Step 3 [Terminal 1]: Commit the update, releasing the locks.

```

COMMIT TRANSACTION;

```

Step 4 [Terminal 2]: Re-read the data. The query now reflects the committed doubled prices for Leyton, illustrating a *Non-Repeatable Read* that READ COMMITTED does not prevent.

```
SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
WHERE town = 'London' AND OldNew = 'Y'
GROUP BY locality;
COMMIT TRANSACTION;
```

READ COMMITTED ensures that only committed data is ever visible, preventing *Dirty Reads*. It does not, however, prevent *Non-Repeatable Reads*.

6.3 Scenario 3 - SERIALIZABLE: Maximum Isolation

Step 1 [Terminal 2]: Begin a read transaction under SERIALIZABLE. Range locks are acquired on the relevant rows in ShortPricePaidData2025.

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN TRANSACTION;
SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
WHERE town = 'London' AND OldNew = 'Y'
GROUP BY locality;
```

Step 2 [Terminal 1]: Attempt to insert a new London property. The statement blocks because the range is locked by Terminal 2's transaction.

```
INSERT INTO ShortPricePaidData2025 (
    TransactionID, Price, DateOfTransfer, Postcode,
    PropertyType, OldNew, PAON, Street, Locality, Town, District, County
)
VALUES (NEWID(), 1000000, GETDATE(), 'E1 6AN', 'T', 'Y',
    '10', 'Brick Ln', 'Spitalfields', 'London', 'London', 'Greater London');
```

Step 3 [Terminal 2]: Re-read the data. The result is identical to Step 1, the blocked insertion from Terminal 1 has not affected the dataset. Committing releases the locks.

```
SELECT locality, AVG(price) AS average_new_property_price
FROM ShortPricePaidData2025
WHERE town = 'London' AND OldNew = 'Y'
GROUP BY locality;
COMMIT TRANSACTION;
```

SERIALIZABLE is the strictest isolation level. It places range locks on the scanned data, preventing any concurrent insertions or modifications to that range until the transaction completes, thus eliminating *Dirty Reads*, *Non-Repeatable Reads*, and *Phantom Reads*.

Declaration

We used Gemini to validate SQL Server syntax during the development of this report. Additionally, we used GitHub Copilot (GPT-5 mini) to assist editing LaTeX formatting.